



Algoritmos e estrutura de dados



Conceitos

Virgínia Mara Cardoso

Vamos refletir?

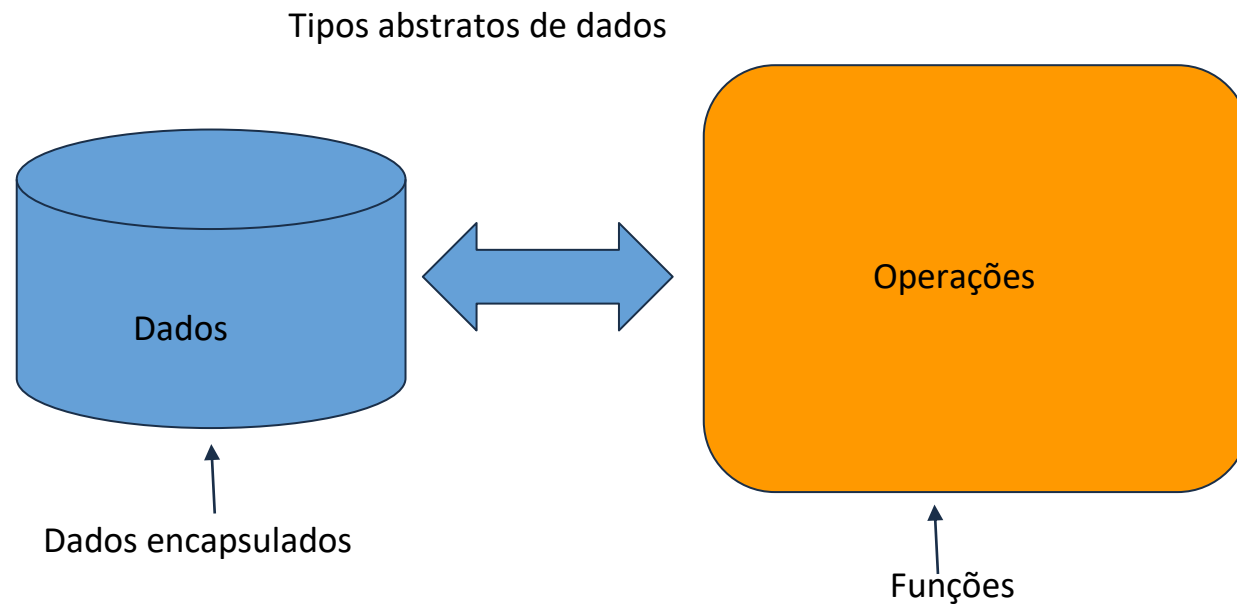
Algoritmos, dados... Trabalhar com eles é fantástico até que deparamos com um programa lento, nem sempre trazendo o resultado esperado, que não atende às nossas expectativas... Você já pensou em como os problemas vão surgindo com o aumento de dados que estamos trabalhando? Será que temos opções ou ideias para lidar com esses problemas?



➤ Tipos abstratos de dados

- O que são TADS?

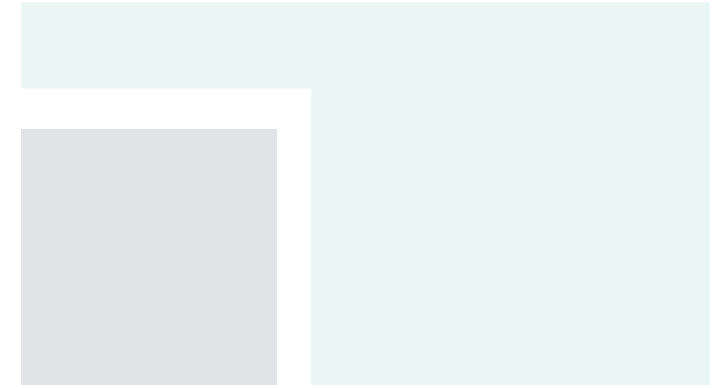
Figura 1 – Tipos abstratos de dados



Fonte: elaborada pela autora.

➤ Tipos abstratos de dados

- **Vantagens:**
 - Função - implementada em vários programas, não sendo específica para um programa.
 - Módulos - independentes do corpo do programa. Podem ser compilados separadamente.
 - Verificação de erros, mais simples e também torna a correção mais rápida e direta. Códigos direcionados.



► Tipos abstratos de dados

- **Como implementar?**
 - Dados e operações.
 - Em linguagens orientadas a objetos:
 - TADS -classes: Atributos (dados) e Métodos (operações).
 - Outras linguagens:
 - Estruturas (dados) e funções (operações).

OBS.: A implementação deve satisfazer as características do TADS, o encapsulamento, a abstração e polimorfismo.



➤ Tipos abstratos de dados

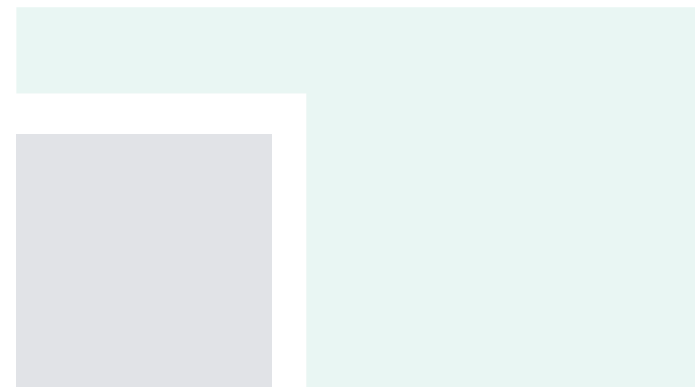
Exemplo: criar um vetor.

// Definindo um VET vetor

```
VET vetor {  
  inteiro tamanho  
  inteiro capacidade  
  elemento[] elementos  
}
```

// Função para criar um vetor

```
função criar_vetor(inteiro capacidade_inicial) -> VET vetor {  
  vetor.vetor <- novo elemento[capacidade_inicial]  
  
  vetor.tamanho <- 0  
  
  vetor.capacidade <- capacidade_inicial  
  
  retornar vetor  
}
```



➤ Tipos abstratos de dados

Exemplo: funções para manipular um vetor.

// Função para verificar o tamanho do vetor

```
função ver_tamanho(vetor) -> inteiro {
```

```
  retornar vetor.tamanho
```

```
}
```

// Função para adicionar um elemento ao final do vetor

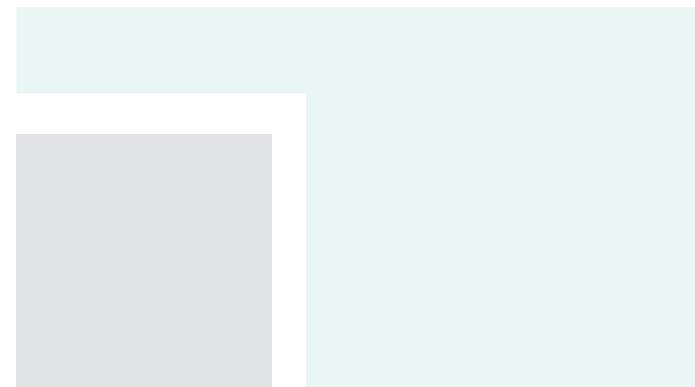
```
função adicionar_elemento(vetor, elemento) {
```

```
  // Adicionar o elemento e atualizar o tamanho do vetor
```

```
  vetor.elementos[vetor.tamanho] <- elemento
```

```
  vetor.tamanho <- vetor.tamanho + 1
```

```
}
```



Listas e filas

Listas e filas

Bloco 2

Virgínia Mara Cardoso



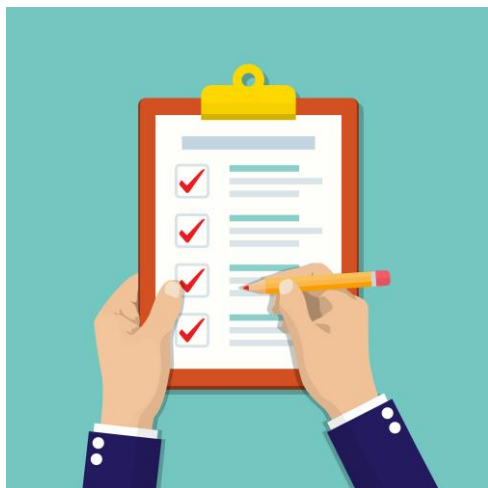


Listas e filas

LISTAS

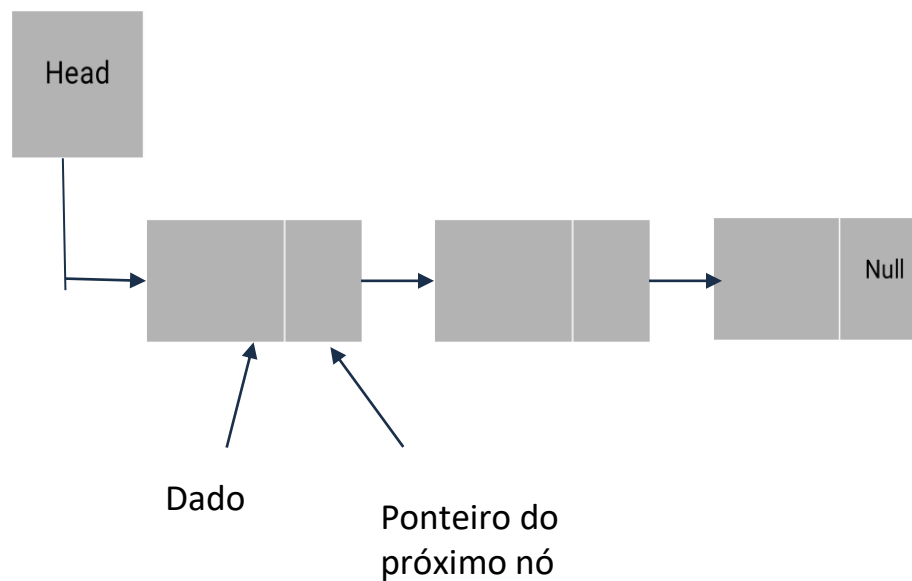
- Estrutura de dados simples - array.
- Armazena os dados em uma forma de lista.

Figura 2 – Lista



Fonte: Shutterstock.com.

Figura 3 – Representação da estrutura de lista

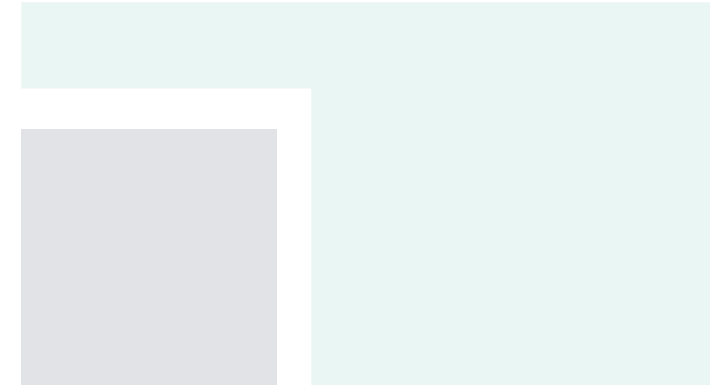


Fonte: elaborada pela autora.

► Listas e filas

Operações com lista:

- Criação da lista.
- Inserção de elemento no início ou final da lista.
- Remoção de elemento no início ou final da lista.
- Consultar elementos na lista.
- Remoção da lista.

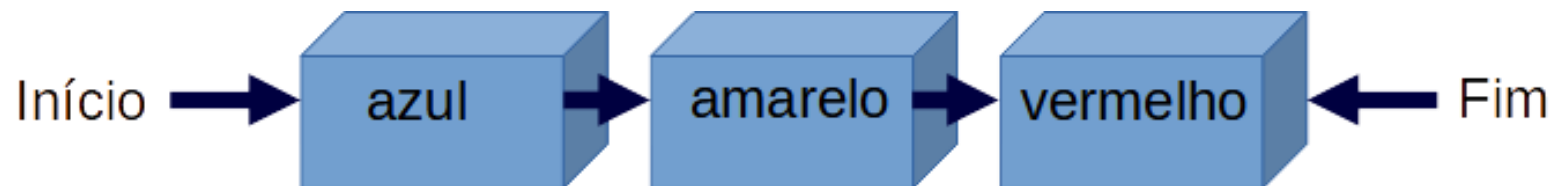


➤ Listas e filas

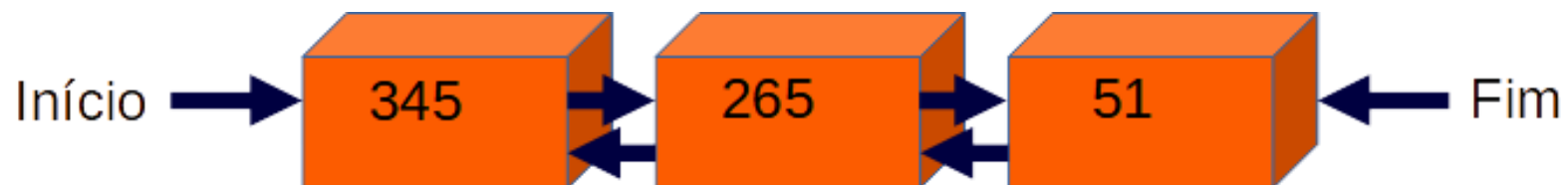
- Tipos de listas:

- Lista encadeada

Figura 4 – Representação de Lista encadeada e Lista duplamente encadeada



- Lista duplamente encadeada



Fonte: elaborada pela autora.



Listas e filas

Filas

- É uma estrutura de dados linear.
- Método FIFO (first in, first out).

Figura 5 – Fila



Fonte: Shutterstock.com.

Figura 6 – Representação de uma fila simples e uma fila circular



Fonte: elaborada pela autora.

► Listas e filas

Operações da fila:

- Criar a fila.
- Inserir elemento, ou enfileirar.
- Remover elemento ou desenfileirar.
- Consultar elementos.
- IsEmpty: verifica se a fila está vazia.
- IsFull: verifica se a fila está cheia.



Pilhas

Virgínia Mara Cardoso

➤ Pilhas

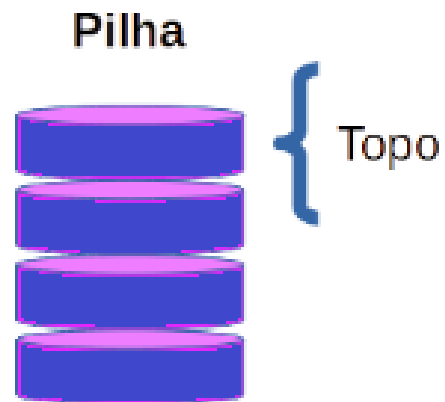
- Estrutura de dados simples - array.
- Armazena os dados em uma forma de lista.

Figura 7 – Pilhas de livros



Fonte: Shutterstock.com.

Figura 8 – Representação de pilha



Fonte: elaborada pela autora.

➤ Pilhas

As operações com as pilhas são:

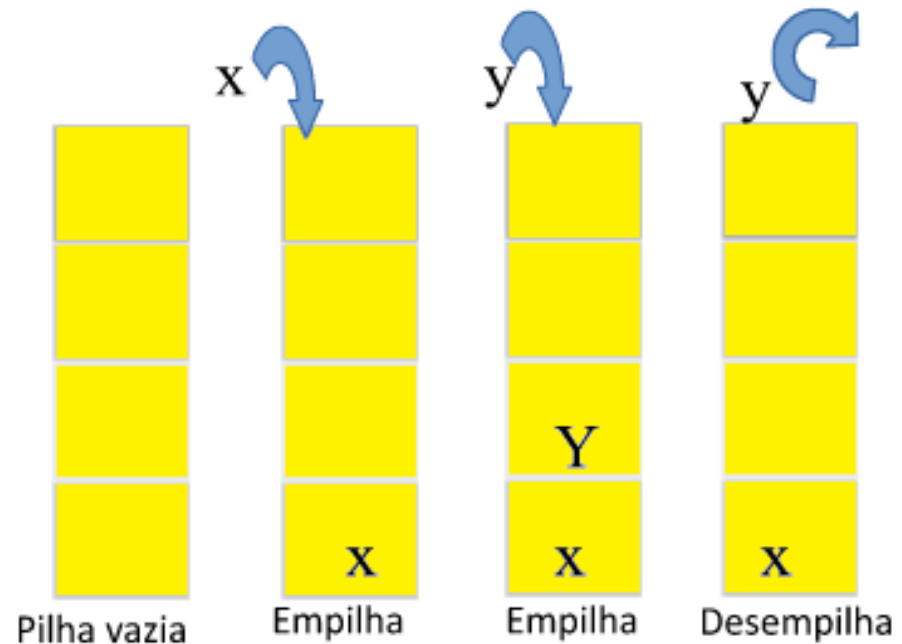
- Criar a pilha.
- Push, ou seja, o empilhar onde você insere, coloca um elemento no topo da pilha.
- Pop, é o contrário do push, aqui você desempilha, retira o elemento do topo da pilha.
- Top: observar o topo, não insere, nem retira, só verifica.
- isEmpty: verifica se a pilha está vazia.
- isFull: verifica se a pilha está cheia.



➤ Pilhas

Utiliza o método LIFO (last in, first out).

Figura 9 – Operações de pilha



Fonte: elaborada pela autora.

➤ Pilhas

Aplicações:

- Editor de texto - fazer e desfazer ações.
- Navegação na internet.
- Resolução de problemas de labirinto.
- Programas de design gráfico - UNDO e REDO.
- Linguagem de programação - análise da sintaxe.



Quiz



Sobre as estruturas de listas, pilhas e filas, você sabe quais métodos elas utilizam para a implementação?

- A** As filas utilizam o método LIFO (Last in, first out).
- B** As listas utilizam o método LIFO (Last in, first out).
- C** As pilhas utilizam o método LIFO (Last in, first out).
- D** O método FIFO (first in, first out) é utilizado para pilhas e listas.

► Quiz - Resolução



Sobre as estruturas de listas, pilhas e filas, você sabe quais métodos elas utilizam para a implementação?

- A As filas utilizam o método LIFO (Last in, first out).
- B As listas utilizam o método LIFO (Last in, first out).
- C As pilhas utilizam o método LIFO (Last in, first out).
- D O método FIFO (first in, first out) é utilizado para pilhas e listas.



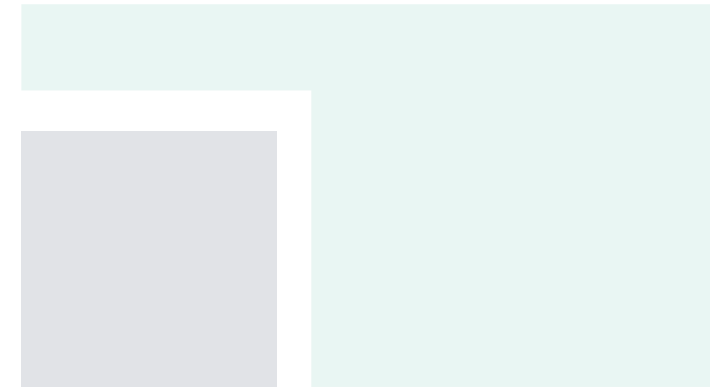


Quiz - Resolução

C) As pilhas utilizam o método LIFO (Last in, first out).

As listas como são estruturas simples não utilizam um método específico, já as filas utilizam o FIFO (first in, first out), o primeiro elemento a entrar é o primeiro a sair.

No caso das pilhas é o método LIFO, o último elemento a entrar é o primeiro a sair.



Teoria em Prática

Teoria em Prática

Bloco 4

Virgínia Mara Cardoso



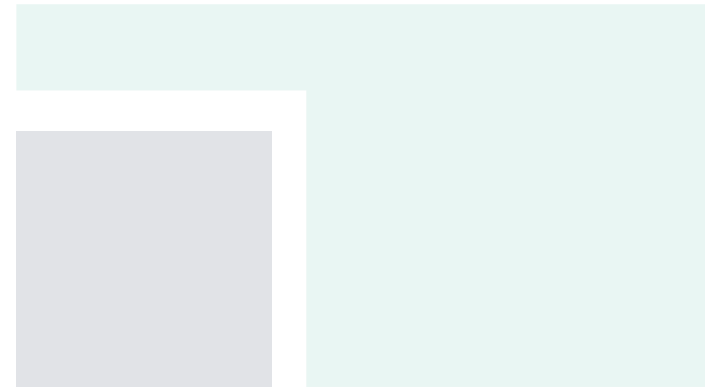
➤ Reflita sobre a seguinte situação

Imagine que você precise carregar vagões em um trem, e estes só podem ser dispostos após o último colocado em uma ordem preestabelecida para a retirada posterior. Mas os vagões chegam e estão ordenados, podendo ser retirados em uma ordem específica. Utilizando a estrutura pilha, vamos tentar resolver?

Figura 10 – Vagões de trem



Fonte: Shutterstock.com.



► Norte para a resolução

- Estruturando o problema:

- o trem: V - vagões

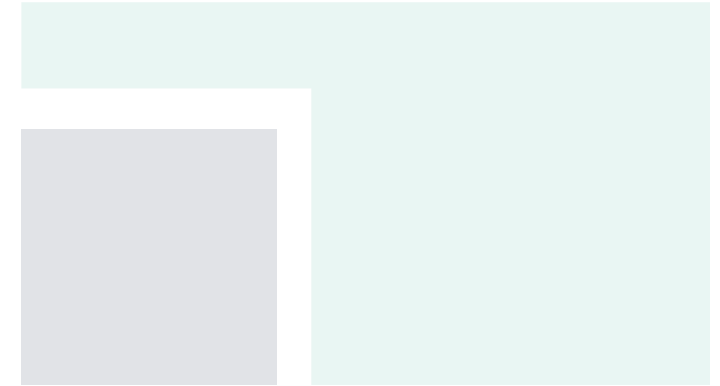
M - máquina

Vagões que devem ser conectados no trem:

$V_s - V_a - V_f$

Devem estar no trem na seguinte ordem:

$V_s - V_f - V_a - M$

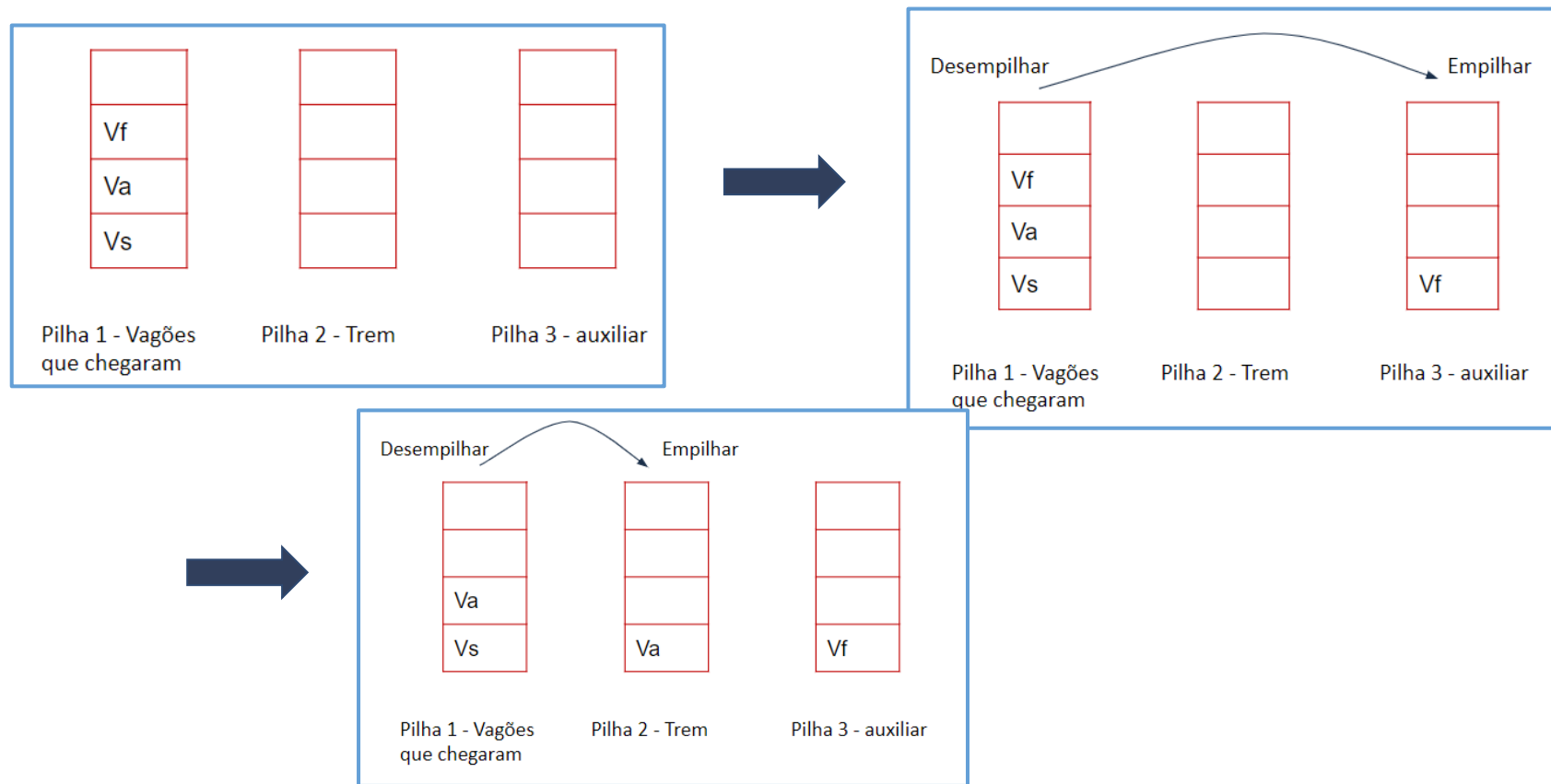


► Norte para a resolução

- Resolvendo o problema:

Estrutura pilha - Método LIFO (last in, first out).

Operações - empilhar e desempilhar.

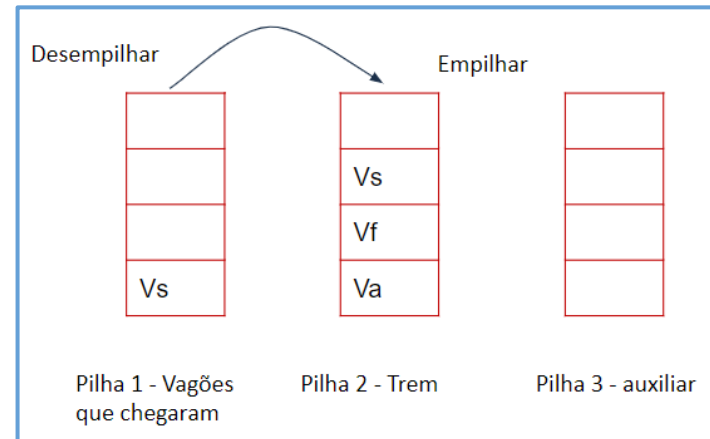
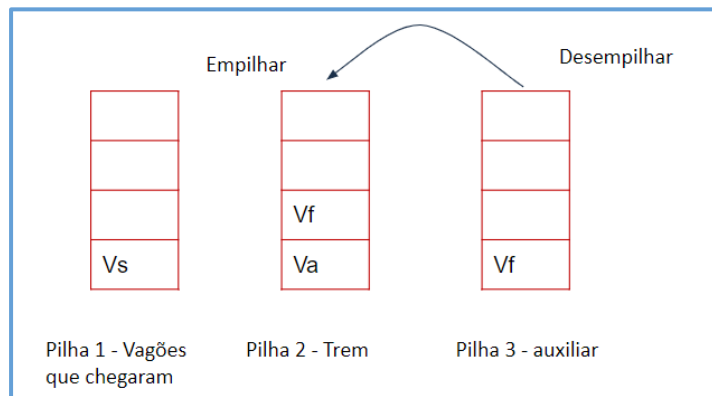


➤ Norte para a resolução

- Resolvendo o problema:

Estrutura pilha - Método LIFO (last in, first out).

Operações - empilhar e desempilhar.

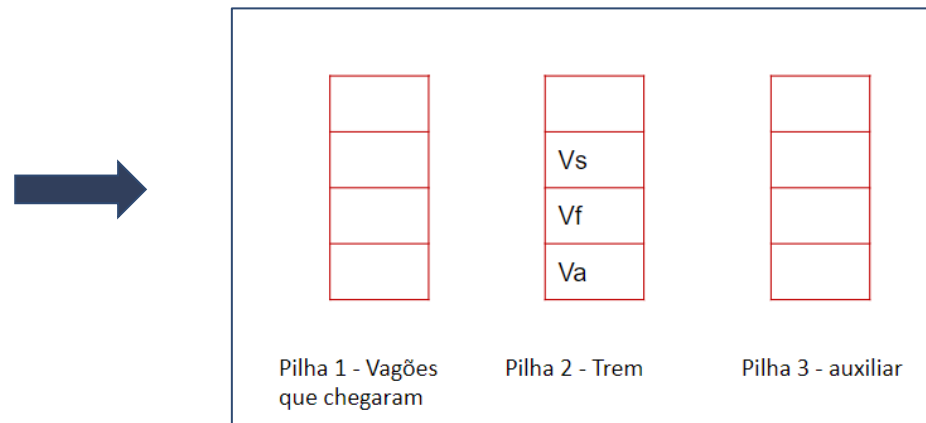


► Norte para a resolução

- Resolvendo o problema:

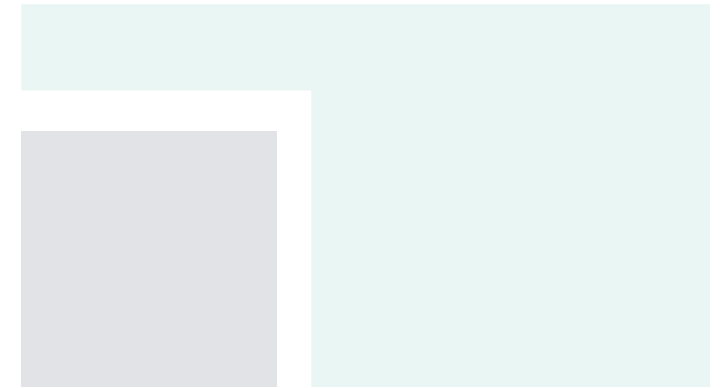
Estrutura pilha - Método LIFO (last in, first out).

Operações - empilhar e desempilhar.



➤ Reflita sobre a seguinte situação

- As estruturas de dados como as listas, filas e pilhas é um grande recurso para trabalhar com dados, para a ordenação e também proporcionar consultas mais rápidas. Imagine que você precise elaborar um algoritmo onde você armazena ações realizadas, como a inclusão de documentos em um sistema. Mas essas ações só podem ser desfeitas ou revisitadas na mesma ordem.
- Como você não sabe o volume de dados de entrada é importante pensar como manipular esses dados e também como ordená-los. Assim, pense nas estruturas de dados que você conhece agora e como elas poderiam auxiliá-lo.





Norte para a resolução

Passos iniciais:

- Defina qual estrutura de dados você deve usar. Assim, desenvolva um algoritmo e faça um teste com um conjunto pequeno de dados e veja se consegue o resultado esperado.
- Nessa etapa sugerimos a consulta dos livros:
 - MANZANO, J. A. N. G. **Algoritmos**: técnicas de programação. 2. ed. São Paulo: Érica, 2015.
 - SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de dados e seus algoritmos**. Rio de Janeiro: LTC, 2020.
- Tente implementar em uma linguagem de programação que você conheça.
- Sugestões para pesquisa:
 - CORMEN, T. H. *et al.* **Algoritmos**: teoria e prática. Rio de Janeiro: Elsevier, 2002.



Dicas do(a) Professor(a)

Virgínia Mara Cardoso



Leitura Fundamental

Prezado aluno, as indicações a seguir podem estar disponíveis em algum dos parceiros da nossa Biblioteca Virtual (faça o login por meio do seu AVA), e outras podem estar disponíveis em sites acadêmicos (como o SciELO), repositórios de instituições públicas, órgãos públicos, anais de eventos científicos ou periódicos científicos, todos acessíveis pela internet.

Isso não significa que o protagonismo da sua jornada de autodesenvolvimento deva mudar de foco. Reconhecemos que você é a autoridade máxima da sua própria vida e deve, portanto, assumir uma postura autônoma nos estudos e na construção da sua carreira profissional.

Por isso, nós o convidamos a explorar todas as possibilidades da nossa Biblioteca Virtual e além! Sucesso!



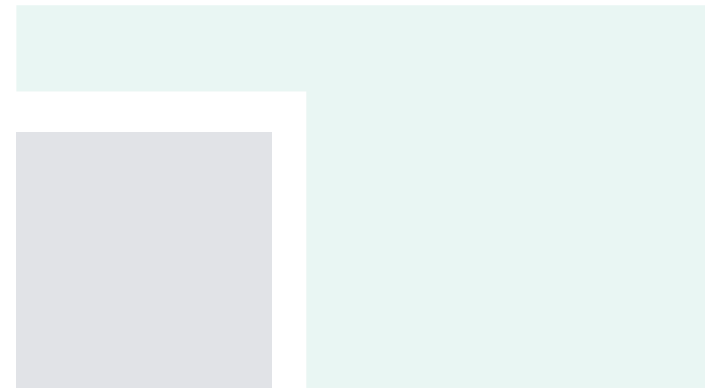


Indicação de Leitura 1

Este trabalho mostra como que alguns algoritmos podem ser mais eficientes se os dados estiverem ordenados.

Referência:

AZEVEDO, R. C. **Análise dos Algoritmos de Ordenação:** buscando a eficiência em sistemas de supermercados. Relatório de pesquisa. Universidade Federal do Amazonas: Instituto de Ciências Exatas e Tecnologia – Itacoatiara.



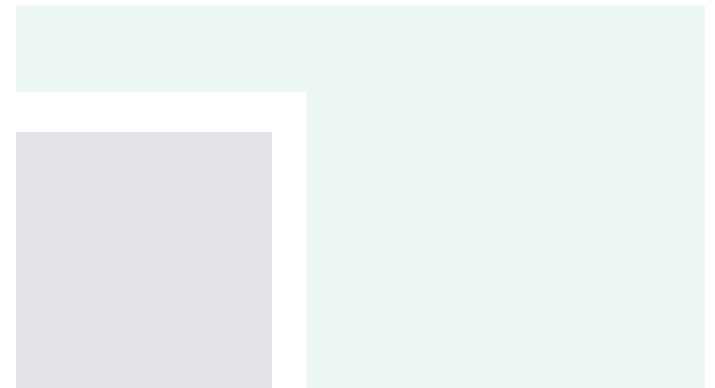


Indicação de Leitura 2

Se você quer conhecer sobre estrutura de dados em Java, este livro é um bom começo.

Referência:

GOODRICH, M. T; TAMASSIA, R.; GOLDWASSER, M. H.
Estruturas de Dados e Algoritmos em Java. Bookman, 2013.





Dica do(a) Professor(a)

Vamos conhecer algoritmos que utilizam estrutura de dados?

Dê uma olhada nos famosos algoritmos de ordenação, você vai gostar e conseguir ver a utilização das estruturas de dados e também conhecer esses famosos algoritmos que tanto nos auxiliam quando temos que trabalhar com muitos dados. São eles: **Selection, Insertion, Bubble, Merge (Sort).**

Referências:

CORMEN, T. H. *et al.* **Algoritmos:** teoria e prática. Rio de Janeiro: Elsevier, 2002.

ZIVIANI, N. **Projeto de Algoritmos:** com implementações em Pascal e C. 2. ed. Cengage Learning, 2009.





Referências

AZEVEDO, R. C. **Análise dos Algoritmos de Ordenação:** buscando a eficiência em sistemas de supermercados. Relatório de pesquisa. Universidade Federal do Amazonas: Instituto de Ciências Exatas e Tecnologia – Itacoatiara. Disponível em: <https://riu.ufam.edu.br/handle/prefix/5136>. Acesso em: 4 maio 2023.

CORMEN, T. H. *et al.* **Algoritmos:** teoria e prática. Rio de Janeiro: Elsevier, 2002.

GOODRICH, M. T; TAMASSIA, R.; GOLDWASSER, M. H. **Estruturas de Dados e Algoritmos em Java.** Bookman, 2013.

MANZANO, J. A. N. G. **Algoritmos:** técnicas de programação. 2. ed. São Paulo: Érica, 2015.

PINTO, R. A. *et al.* **Estrutura de dados.** Porto Alegre: SAGAH, 2020.



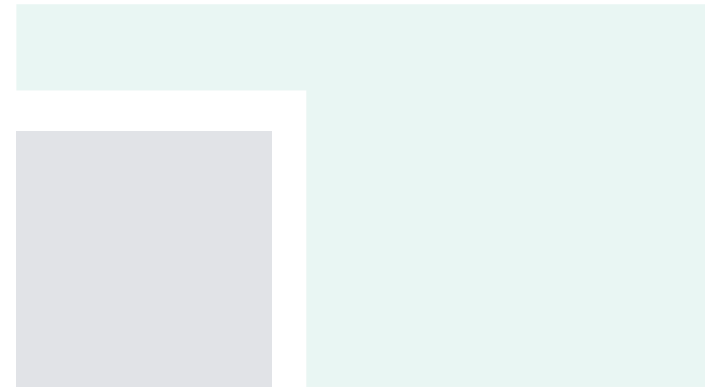


Referências

SOUZA, M. A. *et al.* **Algoritmos e lógica de programação**: um texto introdutório para a engenharia. São Paulo: Cengage Learning Brasil, 2019.

SZWARCFITER, J. L.; MARKENZON, L. **Estruturas de dados e seus algoritmos**. Rio de Janeiro: LTC, 2020.

ZIVIANI, N. **Projeto de Algoritmos**: com implementações em Pascal e C. 2. ed. Cengage Learning, 2009.



Bons estudos!

